

Veritas-Medica -- Technical Onboarding Brief

Prepared for: Ravi -- Senior Solution Architect **Prepared by:** Saeed (CTO / Founder) **Date:** 2026-08-09

Repository: <https://github.com/sabry713-prog/Clinic> (private -- access needed) **Working branch:**

[fix/phase-2-endpoint-wiring](#) (7 commits ahead of [main](#))

This document exists so you can form your own judgement without reading 120 commits. It covers what the product is, how it is built, what genuinely works, what only looks like it works, and where I think your experience will move the needle most. Companion documents are listed in §9.

I have tried to be honest rather than flattering. Where something is mocked, unverified or unfinished, it says so.

1. What the product is

A hospital-deployed **clinical documentation and claim-integrity layer for the Saudi market**. Authenticated clinicians use it to work with a patient's existing record -- and, in the newer half, to capture consultations, generate notes, and get insurance claims right before submission.

The defining architectural decision, and the thing worth your scrutiny first:

Reasoning is separated from language. Every clinical assertion is a deterministic query (Cypher against Neo4j, or SQL against Postgres). The LLM is only ever allowed to reword facts it was handed. It is never asked to know anything clinical.

This is enforced by tests asserting object-equality between what the graph returned and what the UI displays -- not by prompt instructions. That is the strongest claim the product makes, and it is architecturally real.

Regulatory frame (drives most design constraints)

The product was specified as **Health IT, non-SaMD** under SFDA MDS-G027. In practice that means generated text must never *interpret* clinical data:

- Allowed: **Creatinine: 138 (Mar), 141 (Apr), 168 (24 May).**
- Forbidden: **Creatinine has risen, suggesting worsening renal function.**

A mandatory blocklist filter is the final gate on all generated text. **Important caveat -- see §6.1:** scope has since drifted past that line and the required sign-off has not happened.

2. A naming/history note, so the repo makes sense

The repo carries **two lineages** that both still live in the tree. This confuses everyone on first read, so:

Lineage	Lives in	What it is
Original spec MVP ("Clinical Copilot" / "Cortex.ai")	apps/ , packages/ , docs/architecture , docs/api	Spec-driven, non-SaMD, 6 build slices. Postgres + FHIR + classifier + blocklist. Complete.
Veritas-Medica (current name)	services/ , later apps/web work	Sprints 6-10. Neo4j knowledge graph, NPHIES engine, multi-agent orchestrator, Sully-style UI.

They are one running system -- [apps/core](#) fronts everything -- but they were built under different governing documents, which is why [README.md](#) and [CLAUDE.md](#) describe the product differently. **Reconciling those two documents is a real (small) task I would value your opinion on.**

3. System shape

```

Clinician browser (React/TS, Vite, :3000)
?
?
apps/core -- NestJS/TS, :4000, /api/v1
? single authenticated gateway. OIDC (Keycloak), RBAC,
? patient scope, audit hash-chain, proxies everything below.
? No Python service is browser-reachable.
?
??? apps/narrative      Python/FastAPI :5001 grounded prose + blocklist
??? apps/qa             Python/FastAPI :5002 classify -> retrieve -> synthesize -> blocklist
??? apps/transcription Python/FastAPI :5003 dictation / ambient (faster-whisper large-v3)
??? services/orchestrator multi-agent bus (Scribe, Consultant, Pharmacist, NPHIES, Recept
??? services/veritas-graph NSCRE -- Neuro-Symbolic Causal Reasoning Engine (Neo4j)
??? services/nphies-engine FHIR R4 claims, eligibility, pre-auth

Data: PostgreSQL (relational + FHIR JSONB + pgvector) · Neo4j (PSKG + NPHIES graph)
· S3-compatible object store · append-only hash-chained audit + WORM export
Obs: OpenTelemetry -> Jaeger · structured JSON logs, no PHI

```

Monorepo: pnpm workspaces + Turborepo. Python services use **uv**. **Footprint:** ~29k LOC app code across 5 apps, 3 services, 7 shared packages.

Shared packages (packages/)

audit (hash-chain) · **blocklist** (interpretive-language gate) · **classifier** (rule + model layers) · **fhir-client** (FHIR R4) · **phi-guard** (residency classification, fail-closed) · **retrieval** (chunk/embed/hybrid vector+BM25) · **shared-types** (branded IDs, RBAC map, response envelopes)

4. What actually works today

Verified running end-to-end locally (**docker-compose.dev.yml** + deterministic seed of 50 patients):

Capability	State
Auth (Keycloak OIDC), RBAC, patient scope	Working; out-of-scope access returns 403 PATIENT_OUT_OF_SCOPE , verified live
Aggregated patient view, med reconciliation, record search	Working
Factual narrative with hover-to-source provenance	Working
Factual Q&A, EN / AR / code-switched, deterministic refusal path (~16 ms)	Working -- the headline feature
Shift-change handoff (patient + ward)	Working
Audit hash-chain + tamper detection + WORM export	Working
NPHIES claim readiness, ICD/SBS suggest->confirm, rejection analytics, 1-click pre-auth	Working (stub connector; no real payer link)
NSCRE graph reasoning (DDI, renal dose) with evidence chain	Working -- best thing to demo

Capability	State
Ambient dictation capture	Real transcription; SOAP note still canned (\$5)
Medical interpreter mode, specialty draft templates, patient recap	Working

Tests: ~515 passing. Classifier 73/73; blocklist 107/107 (100 % block, 0 false positives); core 72/72; qa 48/48; narrative 33/33; web 43/43. Classifier EN/AR holdout sensitivity & specificity 1.00/1.00.

5. What looks finished but is not

Please read this section before you form an impression from the UI.

- Eight agent action cards do nothing.** `SullyContext.runAgentAction()` appends a chat message and returns. "Adjust Dosage", "Check formulary tier", etc. are dead buttons. (Tracked as M-1.)
- The scribe's SOAP note is canned.** Dictation is real; the note it produces is a six-stage scripted reveal. `generate_soap_note()` exists in the orchestrator and is **never called**. (H-3b.)
- No evaluation harness exists.** There is no `evals/` directory, no labelled dataset, no scoring code. We have engineering test counts, not measured coding precision, NPHIES false-positive rate, or Arabic scribe WER.
- DSR erasure deletes nothing.** `dssr.service.ts` records requests as `pending`; there is no working erasure in Postgres or Neo4j.
- NPHIES profiles are unverified** (`profiles_verified: false`, correctly surfaced on `/health`). No CCHI certificates or digital-signature handling.
- Checklist state has zero backend** -- 0 references in `apps/core`.

6. Known risks -- where I most want your judgement

6.1 Scope has drifted past the SaMD boundary, knowingly

DDI checking, renal dose alerting, differential-diagnosis prose and dose phrasing were all forbidden by the original governing document as SaMD-boundary-crossing. They were each built after being flagged. This is documented in module docstrings but has **not** passed the CTO + Clinical Advisor + Regulatory Consultant sign-off our own process requires. That sign-off gates any real-patient use.

6.2 PHI egress -- the commercial blocker, not just a technical one

`packages/phi-guard` is well designed (fail-closed, three policies, `allow` mode requires a verbatim acknowledgement string so it can't be enabled by a stray env var). **It is bypassed on two paths:** `services/orchestrator/deepseek_client.py` and `apps/narrative` have zero `phi_guard` references while pointing at `api.deepseek.com`. Full encounter transcripts and graph facts (patient IDs, medication names, eGFR values) cross the border.

Deferred deliberately because the data is synthetic. That reasoning holds only while it is synthetic. Under PDPL, health data is sensitive-tier and cross-border transfer needs SDAIA authorisation -- which we will not get for a China-hosted general-purpose LLM. **Self-hosting is the only viable production path in KSA.**

6.3 LLM abstraction is split

`apps/qa` and `apps/narrative` are well abstracted: a `ModelProvider` Protocol with `StubModelProvider` and an OpenAI-compatible `LocalModelProvider`, PHI guard wired in. Swapping to vLLM / Ollama / self-hosted Llama 3 is a config change:

```
QA_MODEL_PROVIDER=local
MODEL_ENDPOINT_URL=https://llm.hospital.sa/v1
MODEL_NAME=llama-3.3-70b-instruct
PHI_INKINGDOM_HOSTS=llm.hospital.sa
```

`services/orchestrator/deepseek_client.py` has **none** of it -- no Protocol, no switch, no guard, `https://api.deepseek.com` hardcoded. All five agents route through it. So the newest, most demo-visible half is the half that cannot move off DeepSeek without code changes. Porting it (?4 days) closes the PHI gap and the lock-in in one change; it is the single highest-leverage item on the board.

6.4 Scaling limits -- bounded and documented

Limitation	Location	Impact
In-process task queue	<code>services/nphies-engine/tasks.py</code>	No survival across restart, no multi-replica
In-process SSE broker	same	Multi-replica drops events for clients on other pods
In-process agent bus	<code>services/orchestrator/agent_buses.py</code>	Same; no durable replay
Free-text medication merge key	<code>etl_pskg.py</code>	"warfarin 3mg" ? "warfarin" -- a dose-suffixed name silently misses its own contraindication edges

The migration path (Celery/Redis or a Postgres outbox) is named in-code. The medication key is the one with clinical consequence.

6.5 Open regulatory / domain questions

- **ICD-10-AM vs ICD-10-CM for NPHIES submission.** The codebase uses AM

throughout (`icd10am_code`, `app.snomed_icd10am_map`, system URI `http://hl7.org/fhir/sid/icd-10-am`). Public sources also reference CM. The two are not interchangeable and a wrong system URI is a rejection cause. **Not guessed -- deliberately left open.**

- Alternative-medication screening has **no therapeutic-class filter** (no ATC

data in the graph). Screening a flagged Metformin can return Warfarin. Payload carries explicit **limitations**, but the output is not demo-safe unless those are read aloud.

- Dictation **speaker attribution is assumed**, not diarised -- lines are labelled "clinician" because that is who pressed record.
- Secrets live in `.env` (gitignored, verified untracked). No Vault/KMS.
- 44 system prompts live inline in Python while `docs/prompts/` documents 12 -- the docs describe the prompts rather than being their source of truth.

7. Planned engineering work (summary)

Full detail with acceptance criteria: [docs/ENGINEERING_WORK_BREAKDOWN.md](#). Effort is person-days for an engineer familiar with the codebase (x1.6 for someone new).

NOW -- 28 days

ID	Item	Days
E2	Port orchestrator to ModelProvider -- closes PHI gap + lock-in	4
E4	Finish UI wiring -- agent cards (2d), SOAP generation (3d), checklist persistence (1d)	6
E1	Clinical accuracy evaluation pack -- harness + labelled datasets	12
E8	Data erasure (PDPL), Postgres + Neo4j	5
E5	ICD-10-AM vs CM discovery spike	1

NEXT -- 31 days: claim-integrity pilot package (15d), NPHIES conformance (8d + external), ATC therapeutic-class data (4d), localisation quick wins -- Hijri dates, Arabic patient templates (4d).

LATER -- trigger-driven: durable queues (8d), SMART-on-FHIR launch (15d), dialect ASR (25d+), ambient consent capture (3d, required before any real ambient use), SOC2/HITRUST (60d+).

Two implementation warnings recorded in that document, both worth your eye:

- **E2 must preserve the Sprint-10 degradation fix.** If the new provider raises differently, an absent API key re-breaks the whole post-care package rather than just the prose field.
- **E8's graph deletion must use an explicit label allowlist.** A naive

DETACH DELETE traversal would destroy shared reference vocabulary (**NphiesDrug**, **DoseRule**, ontology **Medication**) that every other patient depends on. This is the one genuinely dangerous item in the plan.

E1's critical path is **dataset labelling by a clinical coder (~8 of the 12 days)** -- engineering cannot compress it. The 6-week NOW window only holds for one engineer if that labelling runs in parallel.

8. Getting it running

```
git clone https://github.com/sabry713-prog/Clinic.git
cd Clinic
git checkout fix/phase-2-endpoint-wiring # not main -- main is 7 commits behind
cp .env.example .env # then fill in secrets
pnpm install
docker compose -f docker-compose.dev.yml up -d
just migrate && just seed
just dev
```

justfile is the task entry point. Ports: web :3000, core :4000, narrative :5001, qa :5002, transcription :5003, Keycloak, Postgres, Neo4j, MinIO, Jaeger, Mailpit under compose.

Set **QA_MODEL_PROVIDER=stub** for a no-key run. The dev seed is deterministic (mulberry32) -- identical data on every machine, 50 patients, 791 retrieval chunks, 60 historical NPHIES claims.

Two things to know before you clone:

- 1 **main** is behind. The working branch is **fix/phase-2-endpoint-wiring**, pushed and in sync with origin.

- 1 My working tree currently has ~73 uncommitted files (ambient, draft, service-request, transcription, web). I will commit and push before you start, or hand them over as a patch -- tell me which you prefer.

9. Document map

Read in this order:

Order	Document	Why
1	docs/PROJECT-STATUS-FULL-2026-07-22.md	Full lifecycle status, day one -> present
2	docs/PROTOTYPE_EVALUATION.md	Independent evaluation: safety, architecture, market, KSA fit
3	docs/ENGINEERING_WORK_BREAKDOWN.md	The planned work, checked against the actual tree
4	docs/MARKET_READINESS_ROADMAP.md	RICE prioritisation, Now/Next/Later, GTM layer
5	docs/STABILIZATION_AUDIT.md	Code-level audit behind the evaluation
6	CLAUDE.md + README.md	The two governing documents (see §2 -- they disagree)

Reference: [docs/architecture/](#) (system shape) · [docs/api/](#) (endpoint contracts, incl. the 15 k-word NPHIES spec) · [docs/data/](#) (schema, FHIR mapping, retrieval index, audit log) · [docs/classifier/](#) (design, rules, evaluation) · [docs/prompts/](#) (12 safety-critical templates) · [docs/ops/runbooks/](#) (9 incident runbooks) · [docs/evidence-pack-e0.md](#) (measured E0 results) · [docs/executive-demo/](#) (deck + live demo guide).

10. Where I would most value your input

In rough order of how much I think it matters:

- 1 ****Is the deterministic-graph / LLM-as-formatter split the right foundation to scale on****, or does it become a maintenance burden as clinical coverage widens? This is the bet the whole product rests on.
- 1 **The multi-agent orchestrator** -- five agents on an in-process bus. Is that the right abstraction at this stage, or over-engineering that should collapse into direct service calls until the load justifies it?
- 1 **Production topology for KSA**: on-prem vs AWS Riyadh vs Oracle Riyadh, with a self-hosted model endpoint. What would you actually deploy?
- 1 **The three in-process brokers** -- is a Postgres outbox enough, or do we go straight to Celery/Redis before the first multi-site pilot?
- 1 **Sequencing**: E2 first is my call (4 days, closes two problems at once). Would you order the NOW phase differently?
- 1 **Anything in §5 or §6 I have mis-weighted** -- under- or over-stated.

Push back freely on anything here. The documents in §9 are the record, not the sales pitch; where you disagree with them, I would rather know.